SKITII TRIALS PLATFORM

Offline Sync & Device-Assigned Patients

Complete task list for offline-first tablet sync, device-scoped patients, and patient transfer across Trial Mobile App, Backend, and Admin Panel.

VERSION
1.0

DATE
September 2026

AUTHOR
Swapnil Sagvekar

CUSTOMER
Yenepoya Hospital

SCOPE
Trial Mobile App (React Native) + Trial Backend (NestJS / GraphQL / MongoDB) + Trial Admin Panel (Next.js)

Goal

Deliver an **offline-first** trial tablet experience:

- Clinicians work **without internet** after login (enrol patients, Week 0 / weekly forms, sessions).
- All clinical writes are stored **locally on the tablet** first, then **automatically synced** to the Trial Backend when connectivity returns.
- Each patient is **assigned to one tablet** (`assignedDeviceId`) within an institution + trial.
- **Admin panel and tablet** can both **transfer** a patient to another device.
- The backend remains the **system of record** — it validates every synced operation; the tablet mirrors rules for offline UX only.

Locked product rules

These decisions are fixed for this workstream:

Rule	Decision
Patient ownership	Each patient has `assignedDeviceId` — the tablet that "owns" them for dashboard / daily care
Registration number	Unique **per device**, not per whole institution (two tablets may use the same reg no on different devices)
Login	Requires internet (OTP / auth + initial hydrate)
After login	PIN unlock and all clinical workflows work offline
Offline session	Allowed if the patient **exists locally** on the tablet (even before that patient has synced to the server)
Sync timing	Automatic when internet returns — push outbox, then pull assignment changes
Before transfer	App **automatically syncs** that patient's pending outbox first; block transfer if sync fails
Target tablet offline after transfer	OK — target receives the patient on **next online pull**
Session history	Sessions keep the `deviceId` where they were **run**; patient **assignment** can change later via transfer
Who can transfer	**Both** Trial Admin panel and source tablet
API / sync protocol versioning	**Not in scope** for now — use additive GraphQL changes only; SQLite schema migrations on app upgrade

---

What we are not building

Approach	Why not
Termux / Node.js on tablet	Not production-safe, not MDM-friendly, heavy ops burden
Embedded NestJS inside the app	Duplicates server logic, security risk, unmaintainable
Direct MongoDB access from tablet	No auth, no validation, exposes credentials
“Sync raw data only” with no server validation	Admin reports and trial integrity break

Architecture overview

Mental model — three layers



Sync directions

Direction	When	Purpose
Push	Outbox drain when online	Send local creates/updates (enrol, weekly, sessions) to server
Pull	After login, after push, when online	Fetch patients assigned to this device; remove patients transferred away; refresh trial config

Two different “device” fields

Field	On	Meaning
`assignedDeviceId`	Patient	Which tablet **owns** the patient **now** (dashboard list, transfer changes this)
`deviceId`	Session	Which tablet **ran** that session (audit history, never rewritten on transfer)

Outbox ordering & dependencies

The sync worker must respect **dependency order** so the server receives valid payloads:

```
1. CREATE_PATIENT          (clientMutationId, localId)
2. SUBMIT_WEEKLY (Week 0)  (dependsOn: localPatientId)
3. START_SESSION           (dependsOn: localPatientId — wait until serverId exists)
4. STORE_PHASE_READINGS
5. END_SESSION / POST_THERAPY / FEEDBACK
6. TRANSFER_PATIENT        (online only — after patient fully synced)
```

Offline session rule: If the patient exists only locally, the tablet allows the session in the UI. The outbox queues the session but the sync worker sends `START_SESSION` only after `CREATE_PATIENT` succeeds and `serverId` is mapped.

Sync error handling

Error type	Examples	Tablet action
Temporary	No network, timeout, 502/503, expired token	Keep `pending`, retry with backoff; refresh token on 401
Already done	Duplicate `clientMutationId`	Treat as success; map `localId → serverId`
Permanent (business)	`WEEK0_REQUIRED`, `SHIFT_WINDOW_CLOSED`, duplicate reg no on same device	Mark outbox `failed`; show clinician message; do not delete local data
Transfer blocked	Active session, sync failed before transfer	Block transfer UI; show reason

Policy: Local DB keeps what the clinician did. Sync failure means **server/admin does not have the data yet** — surface clearly in UI.

Local business rules (mirror Nest for offline UX)

The tablet must enforce these **before** allowing actions offline (server re-validates on sync):

Rule	Source today	Offline behaviour
Week 0 required before session	`assertCanStartSession`	Block Start Session if `!week0Submitted` locally
Not before enrollment date	Backend session service	Compare cached clinical date vs `enrollmentDate`
Not after trial completion	Backend session service	Compare vs `completionDate`
Shift window open + min minutes left	Trial `shiftWindows`	Use trial config cached at login
Registration no unique	Per device	Check local patients on this tablet
One active session per device	Session service	Local flag on tablet

Trial config (shifts, clinical day start hour, duration) must be **downloaded at login** and stored in MMKV or SQLite.

BACKEND TASKS — `Skitii-Trial-Backend`

B1 — Patient model: device assignment

ID	Task	Description	Status
B1.1	Add `assignedDeviceId` to Patient schema	String field on `patients` collection — the tablet that currently owns the patient for daily care and dashboard listing.	☐
B1.2	Mongo migration	Backfill existing patients: set `assignedDeviceId` from earliest session `deviceId` where available, or leave null for manual admin assignment.	☐
B1.3	Index `{ institutionId, trialId, assignedDeviceId }`	Fast tablet list queries filtered by device.	☐
B1.4	Unique index reg no per device	Replace institution-wide unique index with `{ institutionId, trialId, assignedDeviceId, registrationNo }` (sparse if reg no optional). Allows same reg no on two different tablets.	☐

B2 — Tablet patient APIs (device-scoped)

ID	Task	Description	Status
B2.1	`createPatient` sets `assignedDeviceId`	On create, set from authenticated / verified caller `deviceId`. Reject if client tries to assign a different device.	☐
B2.2	`listPatients` filter by device	Tablet query returns only patients where `assignedDeviceId === caller deviceId` (plus existing institution + trial scope).	☐
B2.3	`getPatient` ownership check	Return patient only if `assignedDeviceId` matches caller device (tablet context).	☐
B2.4	Admin list unchanged scope	Admin GraphQL returns institution-wide patients; include `assignedDeviceId` in response for admin UI.	☐
B2.5	Duplicate reg no check	`assertRegistrationNoUnique` scoped to `institutionId + trialId + assignedDeviceId`.	☐

B3 — Idempotent sync (retry-safe mutations)

ID	Task	Description	Status
B3.1	Accept `clientMutationId` on `createPatient`	UUID generated on tablet when enrol happens offline.	☐
B3.2	Idempotent create	If `clientMutationId` already exists for this device → return existing patient document (no duplicate insert).	☐
B3.3	Idempotent weekly submit	Same pattern for `submitWeeklyAssessment` / `submitWeeklyHrv`.	☐
B3.4	Idempotent session mutations	Same for `startSession`, `storePhaseReadings`, `endSession`, post-therapy, feedback.	☐
B3.5	Structured GraphQL errors	Return `TrialErrorCode` in extensions / payload (`WEEK0_REQUIRED`, `SHIFT_WINDOW_CLOSED`, etc.) so the tablet can classify retry vs permanent failure.	☐

B4 — Session & weekly validation (keep authoritative)

ID	Task	Description	Status
B4.1	Keep `assertCanStartSession`	Do not remove server checks: week0, enrollment/completion dates, shift window, min minutes remaining.	☐
B4.2	Session `deviceId` unchanged	Sessions continue to store which device ran the session; independent of `assignedDeviceId` on patient.	☐
B4.3	Week0 flag on server	After week0 weekly submit, set `week0Submitted: true`, `isEditable: false` (existing behaviour).	☐
B4.4	Ordered patient id	Session mutations require Mongo `patientId`. Tablet sends after patient sync maps `localId → serverId` (no backend change required if app orders outbox correctly).	☐

B5 — Patient transfer (admin + tablet)

ID	Task	Description	Status
B5.1	`transferPatient(patientId, toDeviceId)` mutation	Updates `assignedDeviceId` atomically.	☐
B5.2	Tablet authorization	Tablet may transfer only if patient `assignedDeviceId === caller deviceId`.	☐
B5.3	Admin authorization	Admin may transfer any patient in institution + trial scope (`AdminGuard`).	☐
B5.4	Validate target device	Target `toDeviceId` must be registered for same institution.	☐
B5.5	Block active session	Reject transfer if patient has `ACTIVE` session on any device.	☐
B5.6	Audit collection `patient_transfers`	Log `patientId`, `fromDeviceId`, `toDeviceId`, `transferredBy` (admin user id or device id), `transferredAt`, optional `reason`.	☐
B5.7	Query transfer history	Admin: list transfers for a patient. Optional: institution-wide transfer log.	☐

B6 — Pull sync support

ID	Task	Description	Status
B6.1	Device-filtered patient pull	Extend `listPatients` with optional `updatedSince` ISO timestamp **or** add `syncPull(deviceId, since?)` returning assigned patients + ids removed from this device since `since`.	☐
B6.2	Transfer-away detection	Pull response includes patient ids no longer assigned to this device so source tablet can delete local copies after admin/tablet transfer.	☐
B6.3	Trial config in pull	Return trial configuration version / `updatedAt` so tablet knows when to refresh cached shifts and weekly definitions.	☐

B7 — Auth & device verification

ID	Task	Description	Status
B7.1	Verified `deviceId` on tablet context	JWT or request context carries device id verified at login / device registration — not spoofable from arbitrary client input.	☐
B7.2	Guard tablet mutations	Reject create/list/get if device not registered for institution.	☐

B8 — Devices registry (for admin transfer picker)

ID	Task	Description	Status
B8.1	List institution devices	Admin query: known tablet `deviceId`s (`SKT-…`) for institution — from device PIN registry or dedicated devices collection.	☐
B8.2	Optional device label	Human-friendly name (“Ward A Tablet 1”) for admin UI.	☐

B9 — Testing (backend)

ID	Task	Description	Status
B9.1	Unit: reg no unique per device	Same reg no on two devices succeeds; duplicate on same device fails.	☐
B9.2	Unit: idempotent create	Same `clientMutationId` twice returns one patient.	☐
B9.3	Integration: transfer	Transfer patient A from device 1 → device 2; device 1 list empty, device 2 list includes A.	☐
B9.4	Integration: session after week0	Sync rejects session if week0 not submitted on server.	☐
B9.5	Integration: transfer blocked with active session	Expect error + no assignment change.	☐

ANDROID TASKS — `Skitii-Trials-Yenepoya-HNF-APP`

A1 — Local database (SQLite)

ID	Task	Description	Status
A1.1	Add SQLite library	e.g. `@op-engineering/op-sqlite` — production-grade local store for clinical data.	☐
A1.2	Table `patients`	Columns: `localId`, `serverId`, `assignedDeviceId`, all enrol fields, `week0Submitted`, `isEditable`, `syncStatus`, `updatedAt`.	☐
A1.3	Table `sessions`	Local session records linked to `localPatientId` / `serverPatientId`, status, timestamps, phase data references.	☐
A1.4	Table `session_readings`	Or store compressed blob on session row — match existing gzip payload shape for sync.	☐
A1.5	Table `weekly_submissions`	Completed weekly forms pending sync.	☐
A1.6	Table `outbox`	`id`, `type`, `payload` JSON, `clientMutationId`, `status`, `retryCount`, `lastErrorCode`, `lastErrorMessage`, `dependsOnLocalId`, `createdAt`.	☐
A1.7	Table `sync_meta`	`lastPullAt`, `localSchemaVersion`.	☐
A1.8	Schema migration runner	On app upgrade, migrate SQLite schema (required even without API versioning).	☐

A2 — Repository layer (UI does not call GraphQL directly for writes)

ID	Task	Description	Status
A2.1	`PatientRepository`	CRUD on local `patients`; filter dashboard by `assignedDeviceId === myDeviceId`.	☐
A2.2	`SessionRepository`	Local session lifecycle.	☐
A2.3	`WeeklyRepository`	Persist completed weekly submissions.	☐
A2.4	`OutboxRepository`	Enqueue, list pending, mark synced/failed, clear on transfer.	☐
A2.5	Refactor dashboard	`DashboardScreen` reads local patients via repository — not live `listPatients` as primary source.	☐

A3 — Sync worker (automatic push + pull)

ID	Task	Description	Status
A3.1	`SyncWorker` service	Single orchestrator for push then pull.	☐
A3.2	Push: map outbox type → GraphQL	`CREATE_PATIENT`, `submitWeeklyAssessment`, `startSession`, etc. via existing `graphql-request` client.	☐
A3.3	Dependency ordering	Skip item if `dependsOnLocalId` not yet synced; process FIFO within constraints.	☐
A3.4	Map `localId → serverId`	After successful `CREATE_PATIENT`, store Mongo `_id` on local patient row.	☐
A3.5	Session wait for serverId	Do not send session outbox items until patient has `serverId`.	☐
A3.6	Retry policy	Network/5xx: exponential backoff; 401: token refresh then retry; permanent codes: mark failed.	☐
A3.7	Pull after push	Fetch patients for this device; upsert locally; delete transferred-away patients.	☐
A3.8	Triggers	NetInfo online, app foreground, post-login, post-successful push.	☐
A3.9	Background sync	Consider headless task / WorkManager on Android for sync when app backgrounded (optional enhancement).	☐

A4 — Login & hydrate (online)

ID	Task	Description	Status
A4.1	Login requires network	OTP / auth unchanged — online only.	☐
A4.2	Initial pull on login	Download patients assigned to this device + trial config + weekly schedule definitions.	☐
A4.3	Cache trial config	Shifts, clinical day start hour, trial duration → MMKV or SQLite for offline rule checks.	☐
A4.4	Extend `AppHydrator`	Hydrate local DB from server after PIN unlock when online; merge with existing local pending rows.	☐
A4.5	Keep MMKV for	Device id (`getOrCreatePhysicalDeviceId`), token, PIN, UI config, control music catalog.	☐

A5 — Local business rules module

ID	Task	Description	Status
A5.1	`assertCanStartSessionLocal`	Mirror Nest: week0, dates, shift window, min minutes — in `src/lib/` per AGENTS.md.	☐
A5.2	Reg no unique per device	Validate against local `patients` table before enrol save.	☐
A5.3	Unit tests for rules	Match backend test cases where possible.	☐

A6 — Screen refactors (local first, sync later)

ID	Task	Description	Status
A6.1	Enrol patient	Validate schema → save local patient + outbox `CREATE_PATIENT` with `clientMutationId`.	☐
A6.2	Weekly / Week 0	On complete: save to `weekly_submissions` + outbox; set local `week0Submitted = true` after week0. Keep MMKV for in-progress step draft if desired.	☐
A6.3	Dashboard Start Session	Use `assertCanStartSessionLocal`; remove hard dependency on online `validateSessionStart` (optional online pre-check when connected).	☐
A6.4	`LiveMonitoringScreen`	Write session locally + outbox chain instead of immediate mutations.	☐
A6.5	Post-therapy / feedback	Local save + outbox.	☐
A6.6	Patient edit	Local update + outbox `UPDATE_PATIENT` when editable.	☐

A7 — Transfer (tablet, online)

ID	Task	Description	Status
A7.1	Transfer screen	Enter target device id (`SKT-…`) or scan QR; online only.	☐
A7.2	Auto sync before transfer	Run push sync for that patient's outbox; block if any item failed or still pending after retry.	☐
A7.3	Call `transferPatient`	On success: delete local patient + related outbox rows.	☐
A7.4	Handle admin transfer while offline	On next pull: remove patient if no longer assigned to this device.	☐
A7.5	Messages in `messages.ts`	Transfer success, blocked (sync failed), blocked (active session), invalid target device.	☐

A8 — Sync UX

ID	Task	Description	Status
A8.1	Global sync indicator	Pending count in sidebar or header.	☐
A8.2	Per-patient sync badge	pending / synced / failed on dashboard rows.	☐
A8.3	Failed sync screen	List failed outbox items with error code + manual retry.	☐
A8.4	Offline banner	Extend existing network quality / sidebar offline message.	☐

A9 — Types & structure (AGENTS.md compliance)

ID	Task	Description	Status
A9.1	`src/types/syncType.ts`	Outbox status, sync item types, pull response types.	☐
A9.2	`src/types/patientLocalType.ts`	Local patient row shape.	☐
A9.3	DB layer under `src/services/` or `src/db/`	No business rules in screens.	☐
A9.4	Barrel exports	Update `src/services/index.ts`, `src/types/index.ts`.	☐

A10 — Testing (Android)

ID	Task	Description	Status
A10.1	Unit: outbox ordering	Patient create before session.	☐
A10.2	Unit: local rules	Week0 gate blocks session.	☐
A10.3	Integration: enrol offline → sync	Airplane mode enrol → Wi-Fi → patient in admin.	☐
A10.4	Integration: full offline session chain	Enrol → week0 → session offline → sync all.	☐
A10.5	Integration: transfer	Source transfers → source list empty after pull; target receives patient.	☐

ADMIN PANEL TASKS — `Skitii-Trial-Admin-Frontend`

AD1 — Patient list & detail (visibility)

ID	Task	Description	Status
AD1.1	Assigned device column	Show `assignedDeviceId` on patients list.	☐
AD1.2	Filter by device	Dropdown of institution devices to filter patient list.	☐
AD1.3	Patient detail	Display assigned device prominently on patient view page.	☐
AD1.4	GraphQL query updates	Consume admin patient queries that include `assignedDeviceId`.	☐

AD2 — Transfer patient (admin)

ID	Task	Description	Status
AD2.1	Transfer action	Button on patient list row and/or patient detail: “Transfer to device”.	☐
AD2.2	Target device picker	Modal with institution devices list (`SKT-…` + label).	☐
AD2.3	Confirm dialog	Warn if active session (from API pre-check or error handling).	☐
AD2.4	Call `transferPatient` / `adminTransferPatient`	Trial Backend GraphQL mutation.	☐
AD2.5	Success / error toasts	Map `TrialErrorCode` to user-friendly messages.	☐
AD2.6	Refresh list	Reload patient list after transfer.	☐

AD3 — Transfer history

ID	Task	Description	Status
AD3.1	Patient detail section	Table: from device → to device, date, transferred by (admin email or “Tablet”).	☐
AD3.2	Optional institution log	Sidebar page “Transfers” for audit across all patients (if backend provides query).	☐

AD4 — Devices page

ID	Task	Description	Status
AD4.1	Devices list route	e.g. `/devices` under Operations or Setup.	☐
AD4.2	Columns	Device id, label, institution, optional last seen / registered date.	☐
AD4.3	Sidebar nav entry	Add to `menuConfig.ts` when backend query exists.	☐
AD4.4	Link from patient	Click assigned device → devices detail or filter patients by that device.	☐

AD5 — RBAC (when unified nav / permissions land)

ID	Task	Description	Status
AD5.1	Permission `patient.transfer`	Gate transfer action by role.	☐
AD5.2	Permission `devices.read`	Gate devices list.	☐

AD6 — Testing (admin)

ID	Task	Description	Status
AD6.1	E2E: transfer patient	Admin transfers patient from device A to B; verify column updates.	☐
AD6.2	E2E: filter by device	Filter shows only patients for selected device.	☐
AD6.3	E2E: transfer history visible	After transfer, history row appears on patient detail.	☐

CROSS-CUTTING TASKS

C1 — End-to-end flows to validate

Flow	Steps	Pass criteria
Offline enrol + sync	Login → airplane mode → enrol → online	Patient in Mongo with correct `assignedDeviceId`; admin sees patient
Offline week0 + session	Enrol offline → week0 offline → session offline → sync	Server has week0 + session; week0 gate respected
Reg no per device	Same reg no on tablet A and B	Both succeed; duplicate on same tablet fails locally and on sync
Tablet transfer	Sync patient → transfer to tablet B	A loses patient on pull; B gains on pull
Admin transfer	Admin transfers while tablet offline	Tablet applies on next pull
Sync failure	Force `WEEK0_REQUIRED` on server	Outbox marked failed; local data retained; user sees message
Idempotent retry	Kill network after server saves create	Retry does not duplicate patient

C2 — Recommended build order (dependency only)

Backend and Android move together per feature — do not ship tablet offline writes before backend accepts them.

Step	Backend	Android	Admin
1	B1, B2, B3, B7	A1, A2, A3, A4, A6.1 (enrol)	AD1.1, AD1.4
2	B4	A5, A6.2, A6.3 (weekly + gates)	—
3	B4	A6.4, A6.5 (sessions)	—
4	B5, B6, B8	A7, A3.7 (transfer + pull)	AD2, AD3, AD4
5	B9	A8, A10	AD6
6	—	A9 polish	AD5 when RBAC ready

C3 — Documentation & handoff

ID	Task	Description	Status
C3.1	Update Postman collection	New transfer mutation; device-scoped list examples.	☐
C3.2	Update trial database doc	`assignedDeviceId`, `patient_transfers` collection.	☐
C3.3	Tablet ops runbook	What clinicians see when sync fails; when transfer requires internet.	☐
C3.4	MDM note	Offline sync does not replace forced app updates for schema migrations.	☐

Glossary

Term	Meaning
Local DB	SQLite on tablet — source of truth while offline
Outbox	Queue of pending GraphQL operations to run when online
Push sync	Tablet → server (outbox drain)
Pull sync	Server → tablet (patients assigned to device, config refresh)
clientMutationId	UUID per write for idempotent retries
localId	Tablet-generated patient id before server assigns Mongo `_id`
serverId	Mongo `_id` after successful sync
assignedDeviceId	Which tablet owns the patient now
deviceId (session)	Which tablet ran a specific session

Summary

Repo	Task count (checklist items)	Focus
Trial Backend	B1–B9 (~35 items)	Device-scoped patients, idempotent sync, transfer, pull API, validation
Trial Mobile App	A1–A10 (~45 items)	SQLite, outbox, sync worker, local rules, screen refactors, transfer UX
Trial Admin Panel	AD1–AD6 (~18 items)	Assigned device column, transfer UI, devices list, history
Cross-cutting	C1–C3	E2E flows, build order, docs

North star: Clinicians use the tablet normally without internet. Data lands in Mongo automatically when connectivity returns. Patients belong to one tablet at a time but can move via admin or tablet transfer. The server always validates; the tablet never talks to Mongo directly.

End of document.